



OS SECURITY-II

Cryptography Fundamentals

- Transform data (plaintext) to unreadable form (ciphertext) & back to plaintext.
- **Core Components:**
 - **Plaintext (P):** Original message
 - **Ciphertext (C):** Encrypted message
 - **Key (K):** Secret used to encrypt/decrypt
 - **Encryption (E):** $P \rightarrow C$ $C = E(P, K)$
 - **Decryption (D):** $C \rightarrow P$ $P = D(C, K)$
- **Symmetric Cryptography:**
 - Same key used for both encryption & decryption
 - Strong, well-studied algorithms like **AES** are recommended

Public Key Cryptography (PKC) Basics

- Authenticating messages requires sharing secret keys, impractical at large scale
- Example: Microsoft cannot give all users the same key
- PKC Solution: Use 2 keys:
 - Public Key (K_{decrypt}): Shared openly &
 - Private Key (K_{encrypt}): Kept secret
 - Encryption: $C = E(P, K_{\text{encrypt}})$
 - Decryption: $P = D(C, K_{\text{decrypt}})$

Using PKC Safely and Efficiently

- Use in 3 ways
 - **Authentication:** Encrypt with private key, anyone can verify with public key
Example: Microsoft signs an update file. Your computer verifies it using Microsoft's public key.
 - **Secrecy:** Encrypt with public key, only private key can decrypt
Example: You send a secret message to Alice. You use Alice's public key, and only Alice can decrypt it.
 - **Using both:** Encrypt with sender's private key → authenticate → encrypt with recipient's public key → secrecy
 - Requires **two decrypt operations** for the receiver
- **Limitations:**
 - **Computationally expensive:** 100× slower than symmetric cryptography
 - **Key distribution problem:** Public keys must be reliably known
 - **Critical Rule:** THE CRYPTOGRAPHY'S BENEFIT RELIES ENTIRELY ON THE SECRECY OF THE PRIVATE KEY

Cryptography Hash

- **Compute a hash of the data:**

$$S = H(P)$$

P = original message

S = short summary of P

- **Send:**

- Original message P

- Encrypted hash $E(S)$

- **Receiver checks:**

- Decrypt the hash → get S

- Compute hash of received message → get S'

- Compare S and S' :

- If equal → message is **intact**

- If different → message **was changed**

- Data integrity verification
- Password storage (salted hashes)
- Proof-of-work systems (e.g., blockchain, spam prevention)

Cryptography and Operating Systems

- OS Trust
 - If OS is **trusted**: no need to hide data from it.
 - If OS is **untrusted**: it can access keys and decrypt your data.
- **Limited Protection**
 - Encrypt now, keep key off machine → may help if OS is later compromised.
 - Minimize exposure:
 - Decrypt infrequently
 - Dispose of plaintext quickly
 - Don't store key in system except during use
- **Useful OS Cryptography**
 - **One-way encryption**: e.g., password storage (cannot decrypt even if OS compromised)
 - **Data in transit**: protects data sent over network
 - **Data at rest**: protects storage hardware if key is kept elsewhere

At-Rest Data Encryption (Encryption of Stored Data)

- What it is
 - Encrypting data that is stored on disks, USBs, or other storage devices.
 - Purpose: keep sensitive data safe even if someone gains physical access to the storage.
- Encryption process
 - Data is encrypted before it's stored.
 - When the computer needs to use the data, it decrypts it temporarily.
 - After use, the plaintext (unencrypted data) should be removed from memory.
- Full disk encryption
 - Encrypts the entire storage device (disk, SSD, or other media).
 - Can be done in software (e.g., BitLocker, FileVault) or hardware.
 - Key is needed at boot time (like a password or passphrase).
 - Transparent: once unlocked, you don't see the encryption; the system handles it automatically.
- Performance
 - Encryption uses extra computing power.
 - Software encryption can add a small delay (~few % slower).
 - Hardware encryption usually keeps disk speed similar to a non-encrypted drive.

Cryptographic Capabilities

- Traditional capabilities can be forged; cryptography prevents this.
- Trusted entity creates encrypted capability granting access.
- Symmetric key: same key for creator & verifier, good for same system.
- Public key: creator uses private key, verifier uses public key; good for distributed systems.
- Can include extra info: expiration, user ID, usage restrictions.
- Must be strong enough to prevent guessing/fraud.
- Enables secure, verifiable, and transferable access without central lists.

Security in Distributed Systems

Untrusted Challenges in Distributed systems

- An operating system can only control its own machine's resources, which makes security across multiple cooperating machines a major challenge.
- Untrusted Remote Systems
 - Other machines may not follow your security policies.
 - Some may be adversaries impersonating trusted systems.
 - We must still verify credentials and capabilities from others.
- Untrusted Network
 - Communication occurs over networks no machine fully controls.
 - Attackers can forge, replay, alter, delay, or destroy messages.
 - Network interference threatens authenticity and reliability.
- Solution Direction
 - Cryptography is the essential tool for securing distributed systems.
 - But cryptography is hard to implement correctly.
 - Therefore, use well-designed, standardized cryptographic tools, not ad hoc or custom-built solutions.

Trust and the Need for Authentication

- In a distributed system, we can't always be sure our partners follow the same security policies.
- We often have to trust them, much like we trust friends or coworkers in daily life.
- If that trust is broken, we might detect it later and take steps to protect ourselves (change keys, cut off access, etc.).
- But on a network, all we see are bits going in and out, so how can we be sure who sent them?
- That's where authentication comes in: proving that messages really come from who they claim to.
- Authentication helps us know who we're talking to, even if we can't fully control what they do.

How Authentication Works

- Over a network, authentication usually relies on what you know (like a password) or what you have (like a private key).
- Passwords work well when many people need to log in to one system.
- Public and private keys work better when one system needs to prove its identity to many users, such as a web server using HTTPS.
- Passwords must always be protected in transit and encrypted so eavesdroppers cannot steal them.
- Public keys are often used to set up that encryption safely.
- Even with a secure connection, you still have to trust the other side's system not to misuse that connection.
- In summary, authentication confirms identity, not trustworthiness. You still have to decide who to trust.

Why Public Keys are Important in Distributed Systems

- Problems with Usernames:
 - Usernames are not globally unique (short names can collide).
- Mistaken identity can lead to private information going to the wrong person.
- Connecting to multiple servers requires separate accounts on each machine, inconvenient.
- Use public keys as universal identities (called principals).
- People are represented by their public keys, which are unique and domain-neutral.
- Public keys provide a reliable, cross-domain identity for distributed systems, solving problems of uniqueness, convenience, and trust.

Distributed Access Control Lists (ACLs)

- Just like the discussed ACL lists permissions like Read, Write, Execute.
 - Each permission is associated with an identity, typically a public key.
- Security:
 - ACLs are signed by the file owner, so only the owner can modify them.
 - Group permissions can be signed by a group key for shared control.
- Distributed Storage:
 - ACLs can live on different servers from the actual data.
 - Signatures allow verification of ACLs even when stored separately.
 - ACLs can be stored independently from the systems that enforce them.
- Importance:
 - Ensures that permissions are authentic and cannot be tampered with.
 - Supports distributed systems, where data and control may be spread across multiple servers.

Advantages of using Public Key in ACL

- Identity Verification:

- Each user or client has a public/private key pair. Their identity is tied to their public key, and they prove who they are by signing requests with their private key.
- Example: Alice wants to read a file. She signs her request with her private key. The server can check that the signature matches her public key in the ACL.

- Data Access Security:

- Clients cannot request or modify data unless they own the private key corresponding to the public key listed in the ACL. This ensures only authorized users can access the data.

- Server Trust:

- Servers only use ACLs that are signed by the file owner. This prevents a malicious server from altering permissions.

- Cross-domain usability:

- Public keys work across multiple systems — you don't need separate usernames for each server.

Challenges with ACL

- What happens if someone steals your private key?
 - They can now pretend to be you and access files.
 - To fix this, you would need to update every ACL that uses your key. This is very expensive and impractical.
- Better Approach:
 - Assign a unique identifier (like a stable user ID) in ACLs instead of using a key directly.
 - Link that identifier to your current public key via a certificate issued by a Certificate Authority (CA).
 - This way, if your key is compromised, you can just update the certificate rather than every ACL.
- Client Request Process:
 - When requesting access, the client signs (request + unique ID) with their private key.
 - The client may also provide the certificate proving that their public key matches their ID.
- Certificate Revocation:
 - If your key is compromised, the CA needs to distribute a revocation notice so that servers stop trusting the old key.

Another Issue

- Cache Inconsistency Problem
 - Servers may cache ACLs to avoid fetching them every time.
 - If a user is removed from an ACL, a server with a cached ACL might still let them access the file.
 - This is a security risk, because access control is no longer consistent across servers.

Ensuring Data Integrity and Freshness in Distributed Systems

Who signs the data?

- Server signs data: Convenient, but if server is compromised, client can't trust it.
- Owner signs data: Secure, but only the owner can update the file, impractical in many systems.
- Group of servers signs: Safer, but requiring all servers to sign can block updates if even one server misbehaves.

Threshold Signatures

- Instead of all servers, require signatures from a threshold number of servers.
- This balances safety and availability.
- Byzantine agreement protocols can help achieve this securely in the presence of some faulty or malicious servers.

Data Freshness Problem

- A valid signature only proves the data is authentic, not that it is the latest version.
- Freshness attack: Malicious servers may return old data (e.g., payroll update blocked).
- This affects both ACLs and data.

Solutions

- Invalidate old copies after updates.
- Rely on a trusted group of servers to maintain and agree on the most recent version (Byzantine agreement).

Distributed Decision Making & Two-Phase Commit (2PC)

- Why distributed decision making?
 - Fault tolerance: A group of machines can make a decision even if some fail.
 - After a decision, results are recorded in multiple locations for reliability.
- Two-Phase Commit Protocol (2PC)
 - Ensures all machines agree to commit or abort a transaction, even after crashes.
 - Each machine maintains a stable log to track the transaction state.
- Phase 1: Prepare
 - Coordinator asks participants to promise to commit or rollback.
 - Participants record the promise in their log and respond.
 - If anyone votes to abort → coordinator writes Abort in its log and tells everyone to abort.
- Phase 2: Commit
 - If all participants are prepared → coordinator writes Commit in its log.
 - Coordinator tells all nodes to commit → nodes acknowledge → coordinator writes Got Commit.
- Why it works:
 - The log ensures consistency: after a crash, each machine can recover the correct transaction state.
 - Guarantees that all or none of the participants commit.

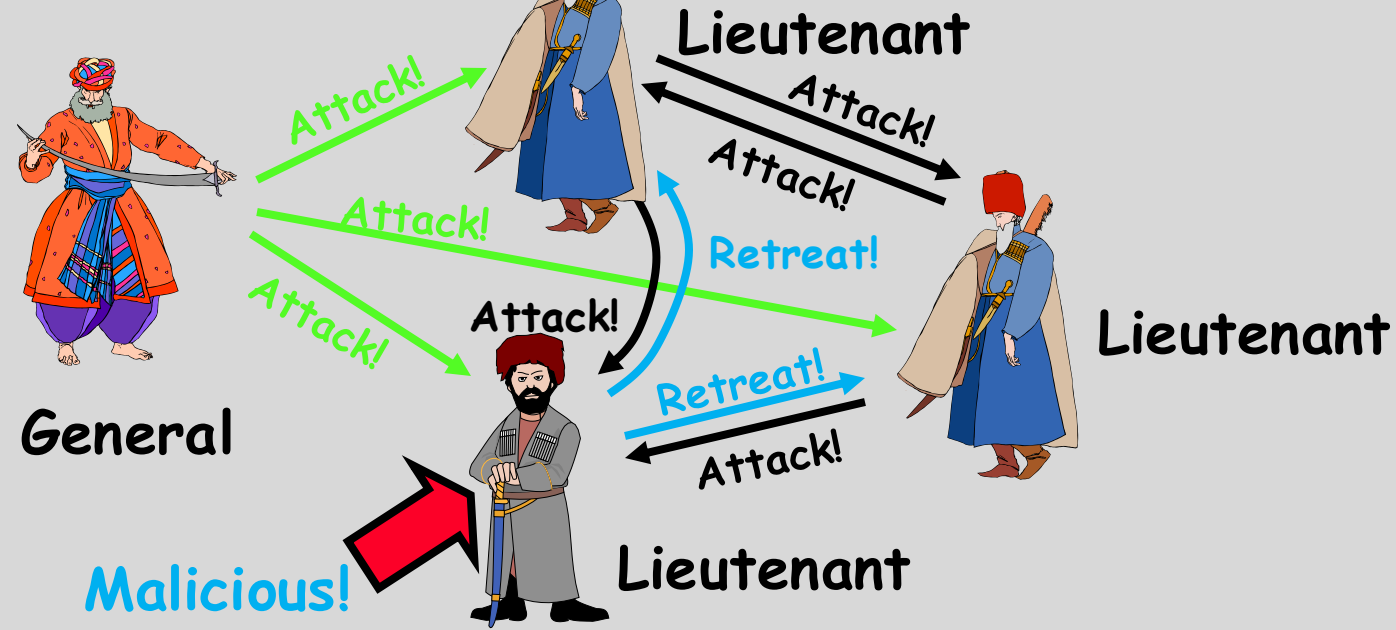
Limitations of Two-Phase Commit (2PC)

- Blocking Problem
 - 2PC can stall a machine if another site crashes.
 - Example:
 - Site B votes “yes” (prepared) and logs it, then crashes.
 - Site A (coordinator) also crashes.
 - When B wakes up, it cannot decide to commit or abort, because it doesn’t know if A committed.
 - Result: B is blocked, holding resources like locks or pinned pages until coordinator recovers.
- Three-Phase Commit (3PC) can avoid blocking by adding an extra phase.
- Security Concern
 - What if one or more nodes are malicious?
 - They may try to manipulate or compromise the distributed decision-making process.

Byzantine Fault

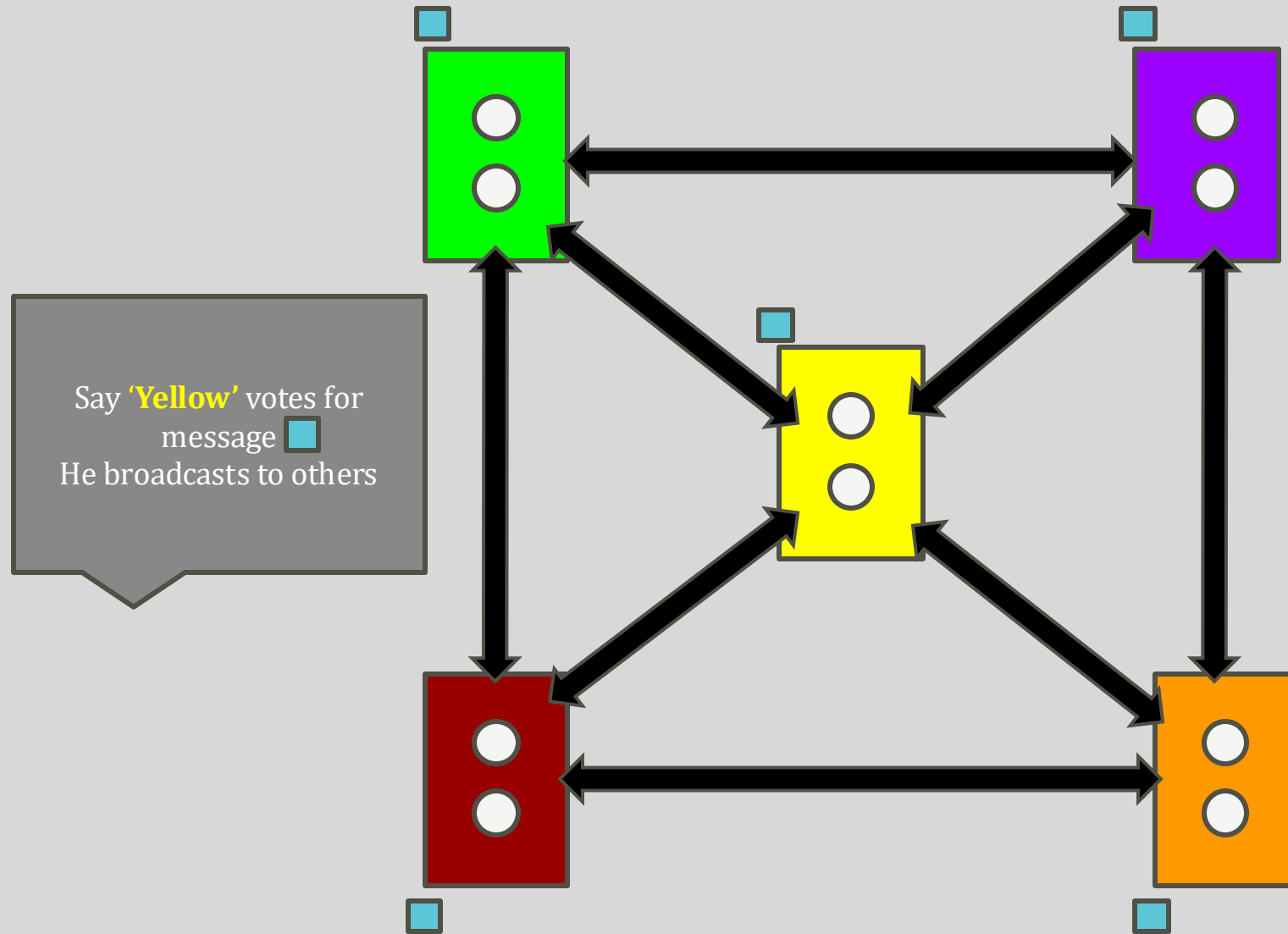


Byzantine General's Problem

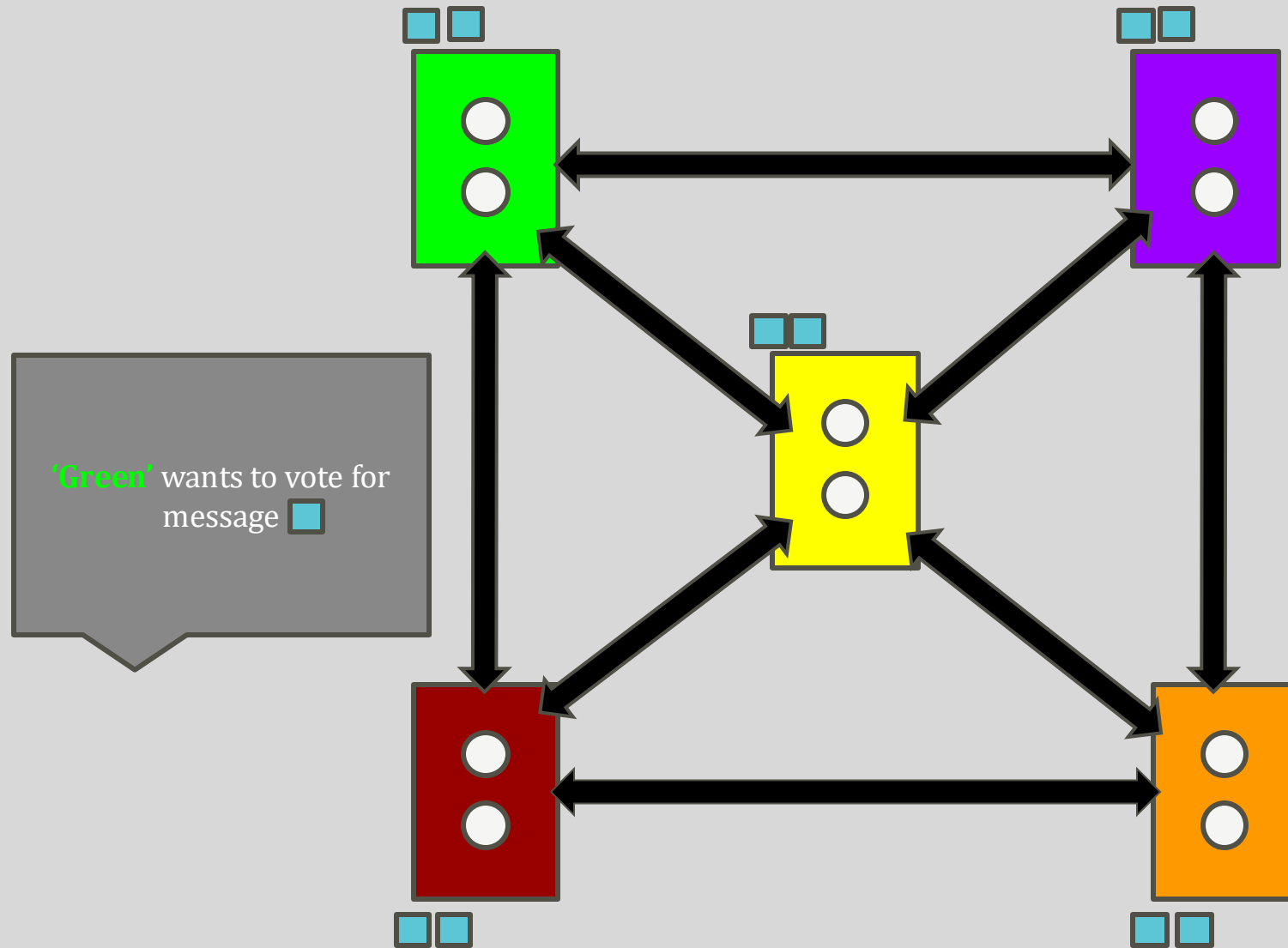


- There is **1 commander (general)** and **n-1 lieutenants**.
 - Some of them (up to f) might be **traitors** (insane or malicious).
 - The goal is to **send an order** so that the loyal ones behave correctly.
- **Rules to satisfy:**
 - **IC1 (Consistency):** All loyal lieutenants must **agree on the same order**, no matter what the traitors do.
 - **IC2 (Validity):** If the general is loyal, then the loyal lieutenants must **follow exactly what the general ordered**.

Byzantine Fault



Byzantine Fault

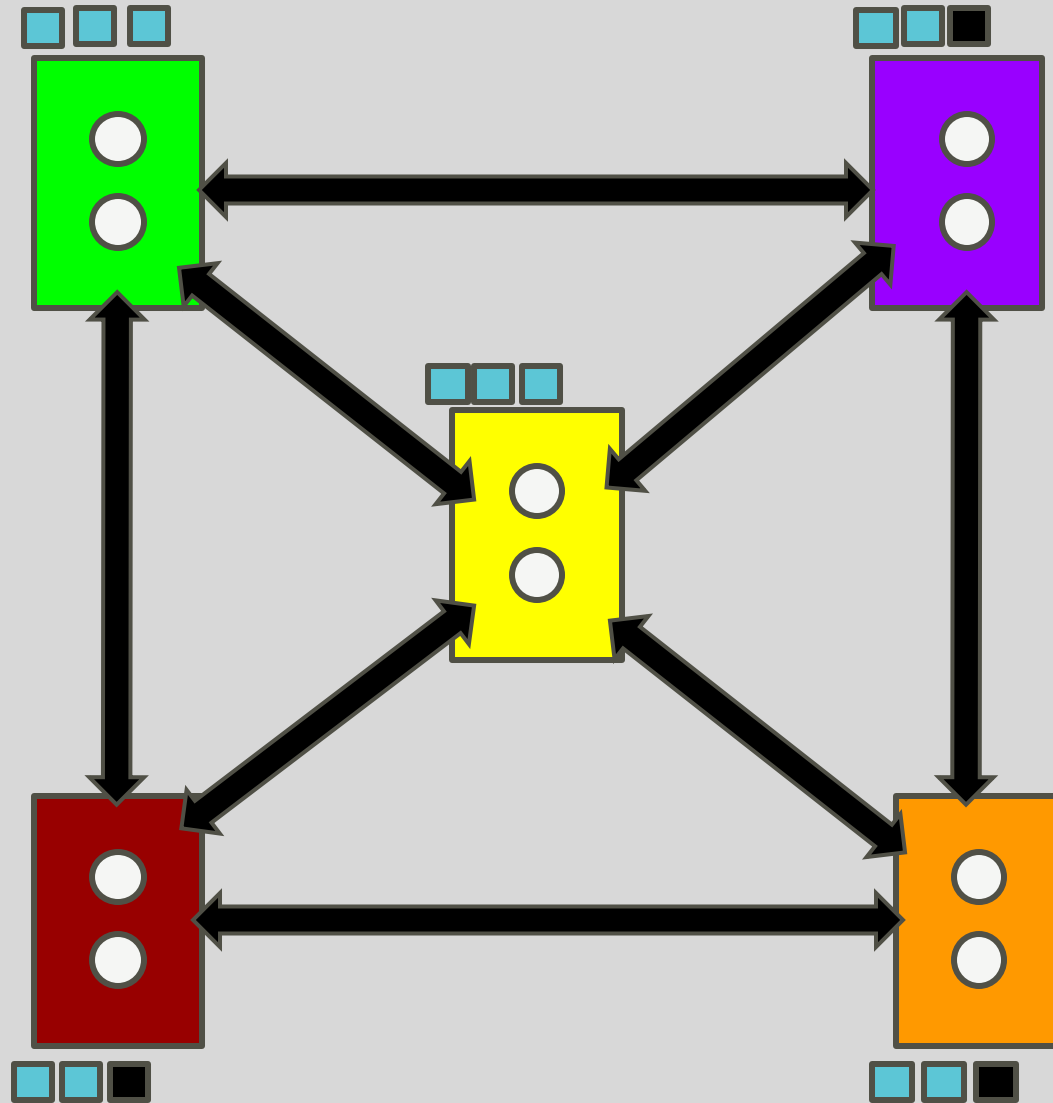


Byzantine Fault

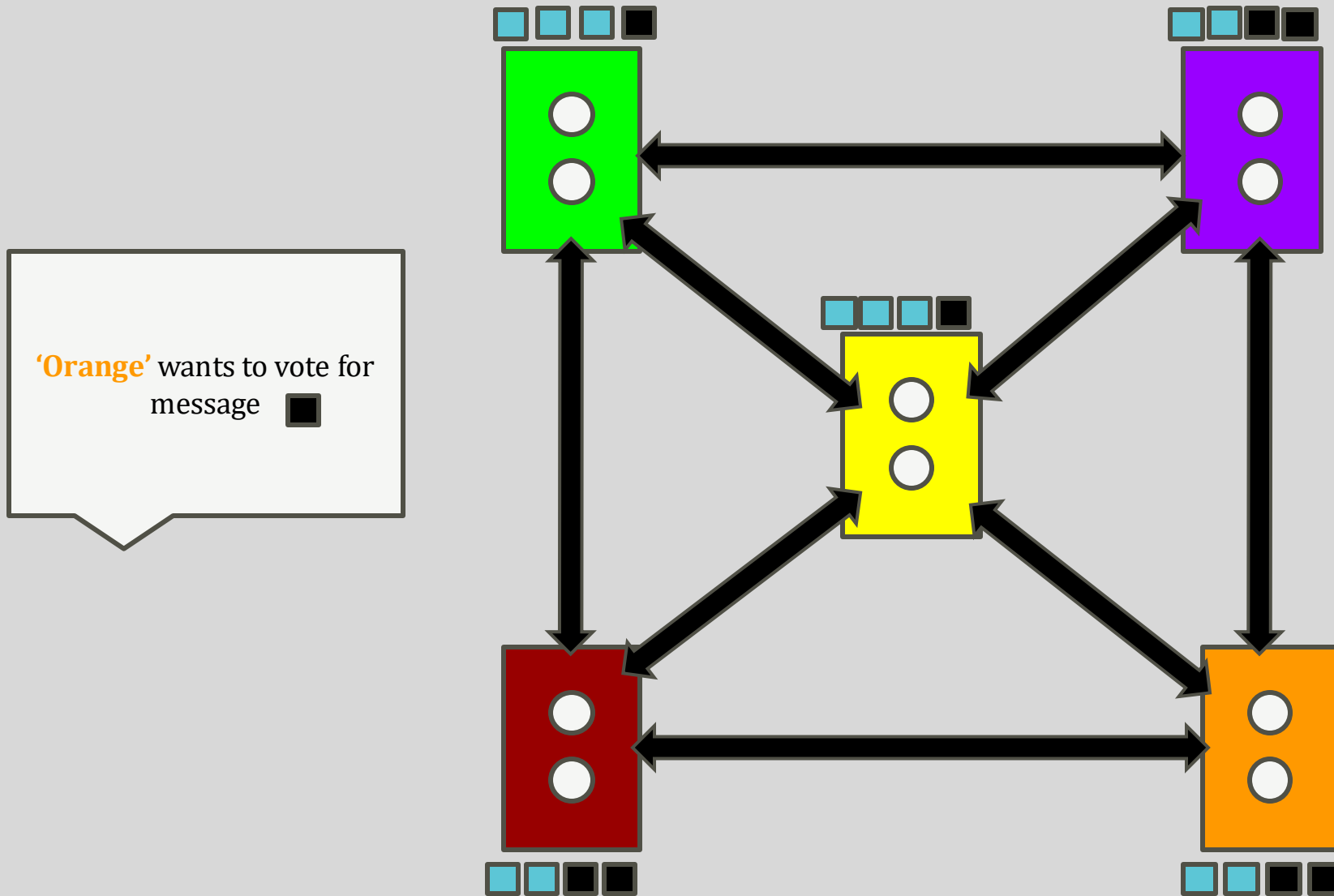
Assume **'Purple'** is a malicious node

'Purple' sends message to 'Yellow' and 'Green'

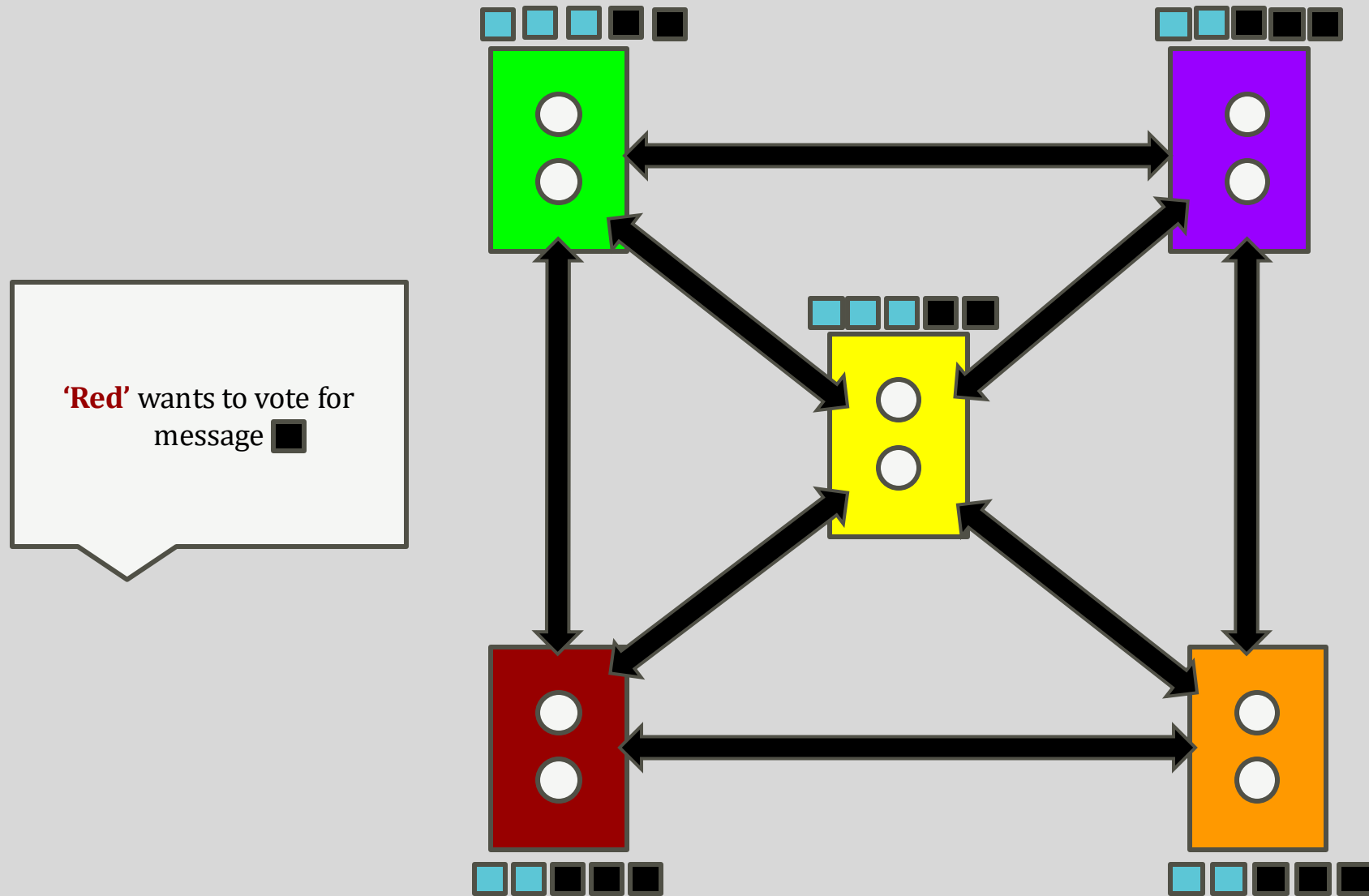
'Purple' sends message to 'Red' a ■ 'Orange' and itself



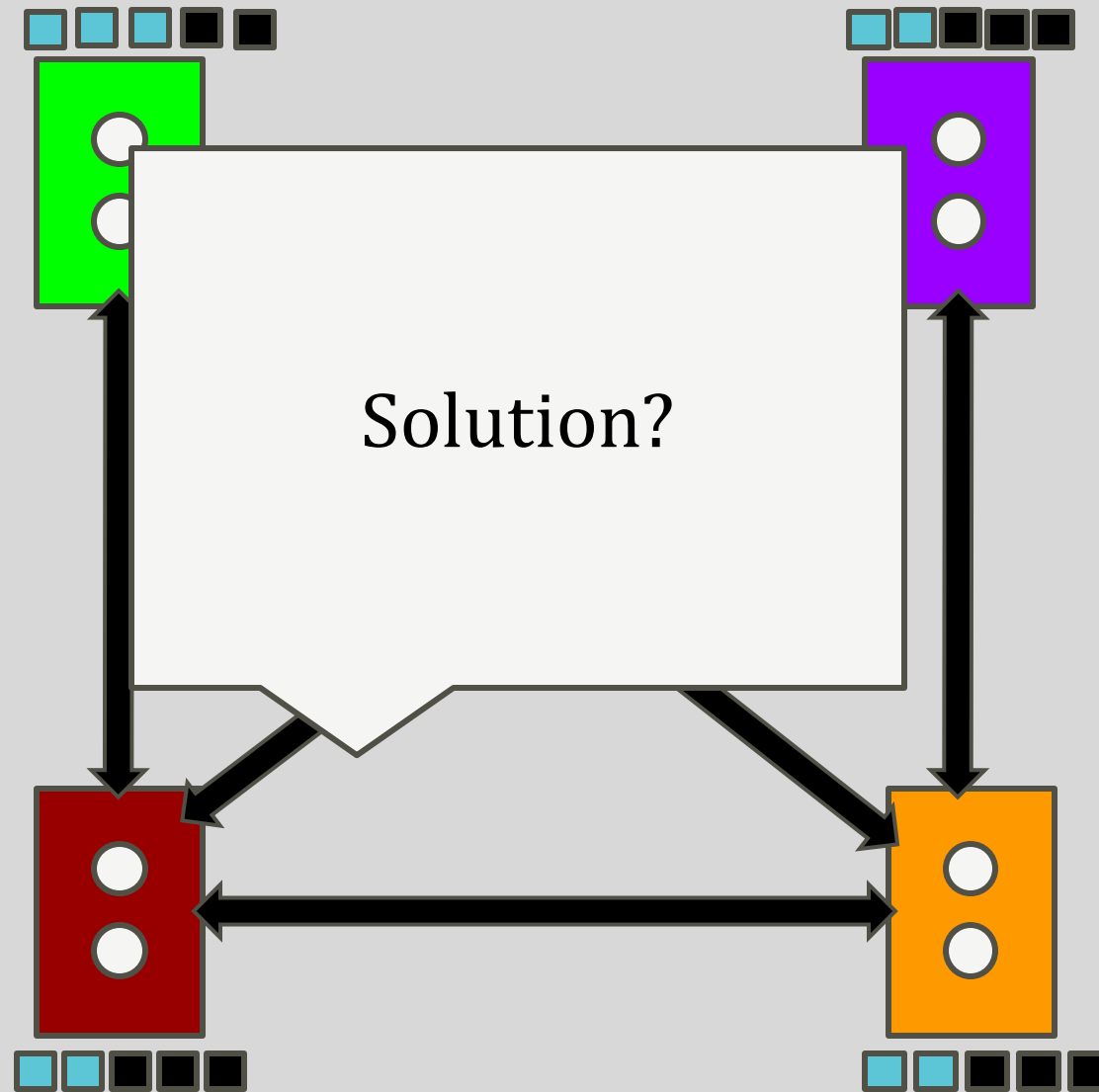
Byzantine Fault



Byzantine Fault



Byzantine Fault



Based on majority voting, the network logically splits

Yellow and Green adds message  to their chain

Purple, Orange and Red adds message  to their chain

This violates the 'Safety' property of a consensus

The ledger will now be inconsistent

Different nodes having different ledgers



Practical Byzantine Fault Tolerance (PBFT) In Blockchain

PBFT (Practical Byzantine Fault Tolerance)

- ❑ It is a consensus algorithm designed to allow a distributed system to reach to a consensus even when some of its nodes are faulty (even malicious)
- ❑ Why the “Practical” word in PBFT?
 - ❑ Because it considers an asynchronous system
 - ❑ Realistic networks are asynchronous in nature
- ❑ What is an asynchronous system?
 - ❑ There is no bound on message delay.
 - ❑ When we have sent a message, we don't know when will it get delivered.

Technical Terms

❑ Client

- ❑ The entity(node) that sends a request(transaction) to the blockchain network.

❑ View

- ❑ A sequence of rounds where a particular primary replica is in charge.
- ❑ A view will change if the primary is suspected to be faulty(behavior)

❑ Replicas

- ❑ Nodes participating in the consensus process

❑ Primary Replica

- ❑ A replica which is designated as the primary leader for a specific view (a round of consensus).

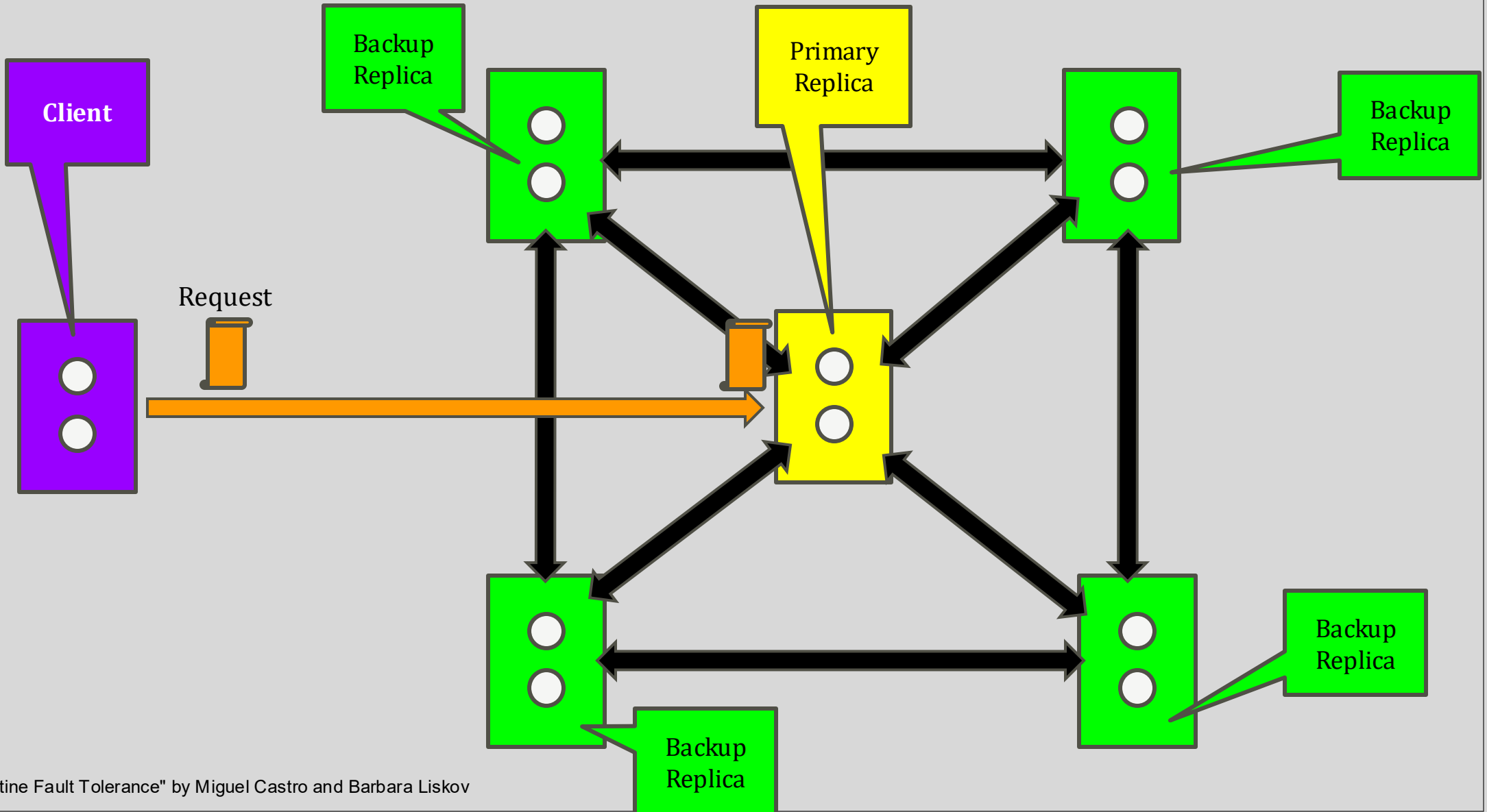
❑ Backup Replicas

- ❑ The other replicas act as backups.

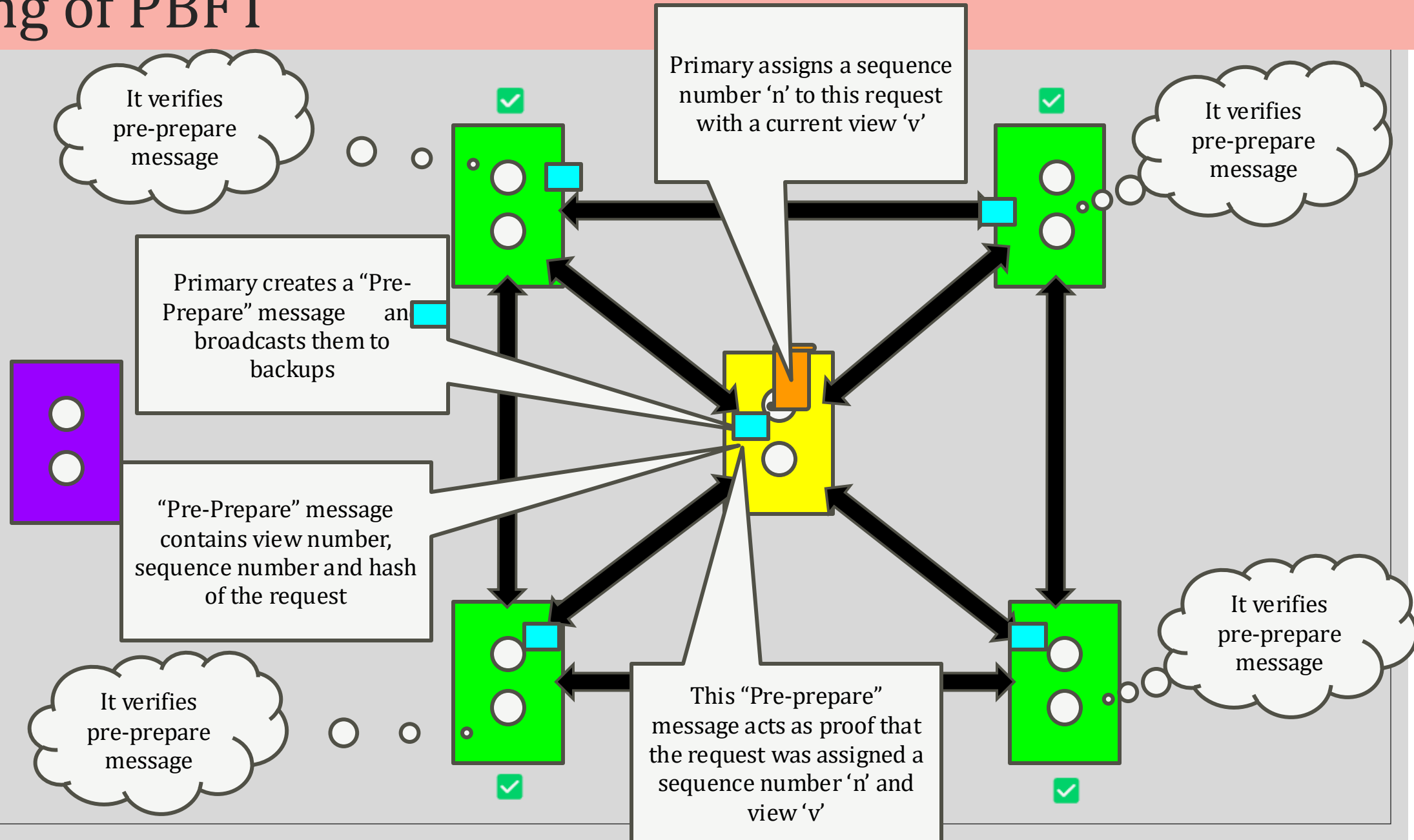
Number of Nodes Requirements in PBFT

- ❑ Assuming there are 'F' faulty nodes in the network:
 - ❑ To tolerate 'byzantine nodes' (we want to reach to a consensus even in the presence of a byzantine behavior), we require at least **"3F+1" total nodes (primary+backup) in the network.**
 - ❑ To be able to reach to a consensus, a minimum of **"2F +1" matching messages** (prepare and commit messages) is required.

Working of PBFT



Working of PBFT

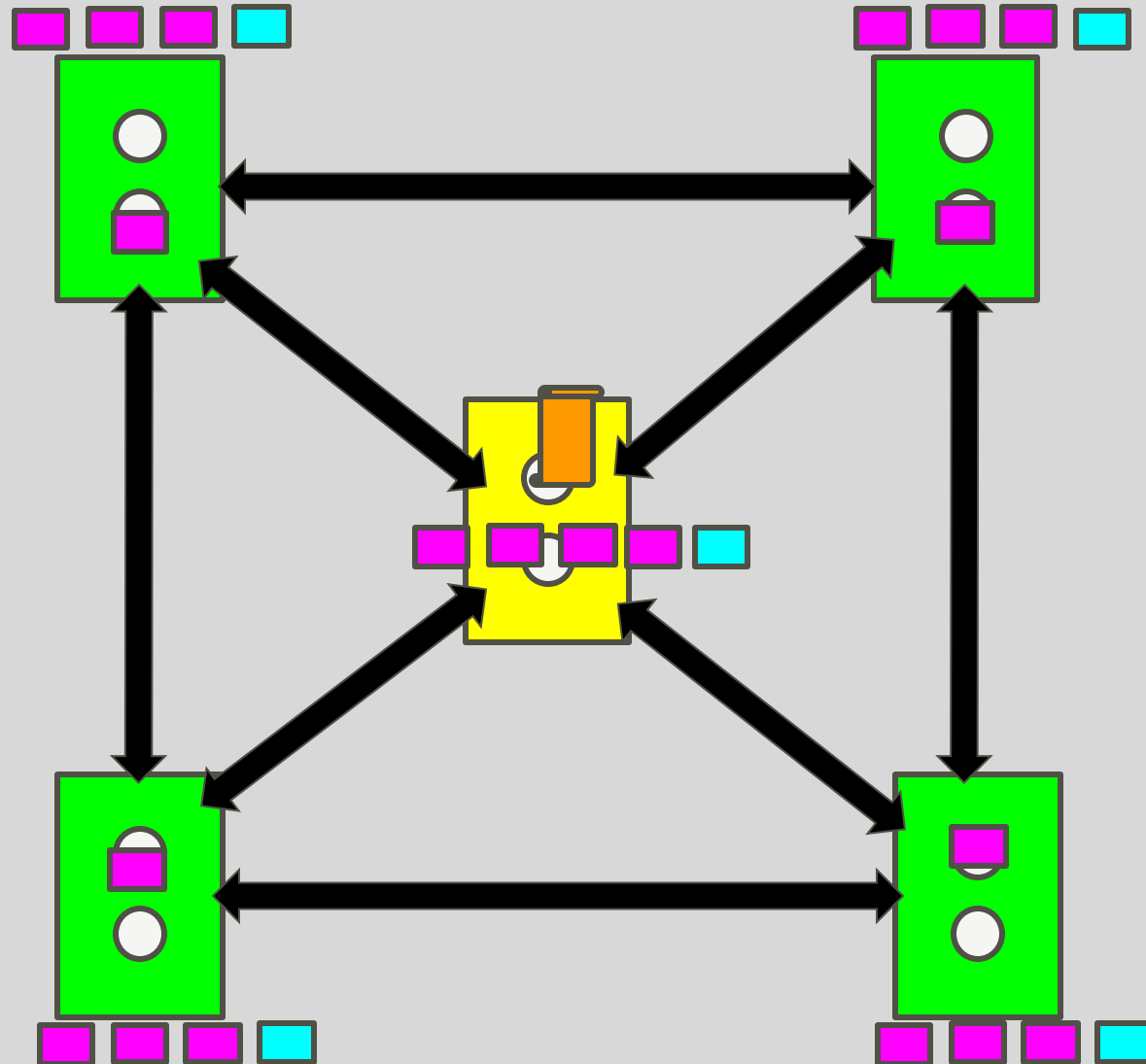


Working of PBFT

After accepting the “pre-prepare message”, backups broadcasts the “prepare message” to other replicas including primary

“Prepare Message” acts as a proof that the backups agreed on the “pre-prepare” message with view number ‘v’ and sequence number ‘n’

Each backups and a primary matches the “Prepare Message” it receives with the earlier “Pre-prepare messages”

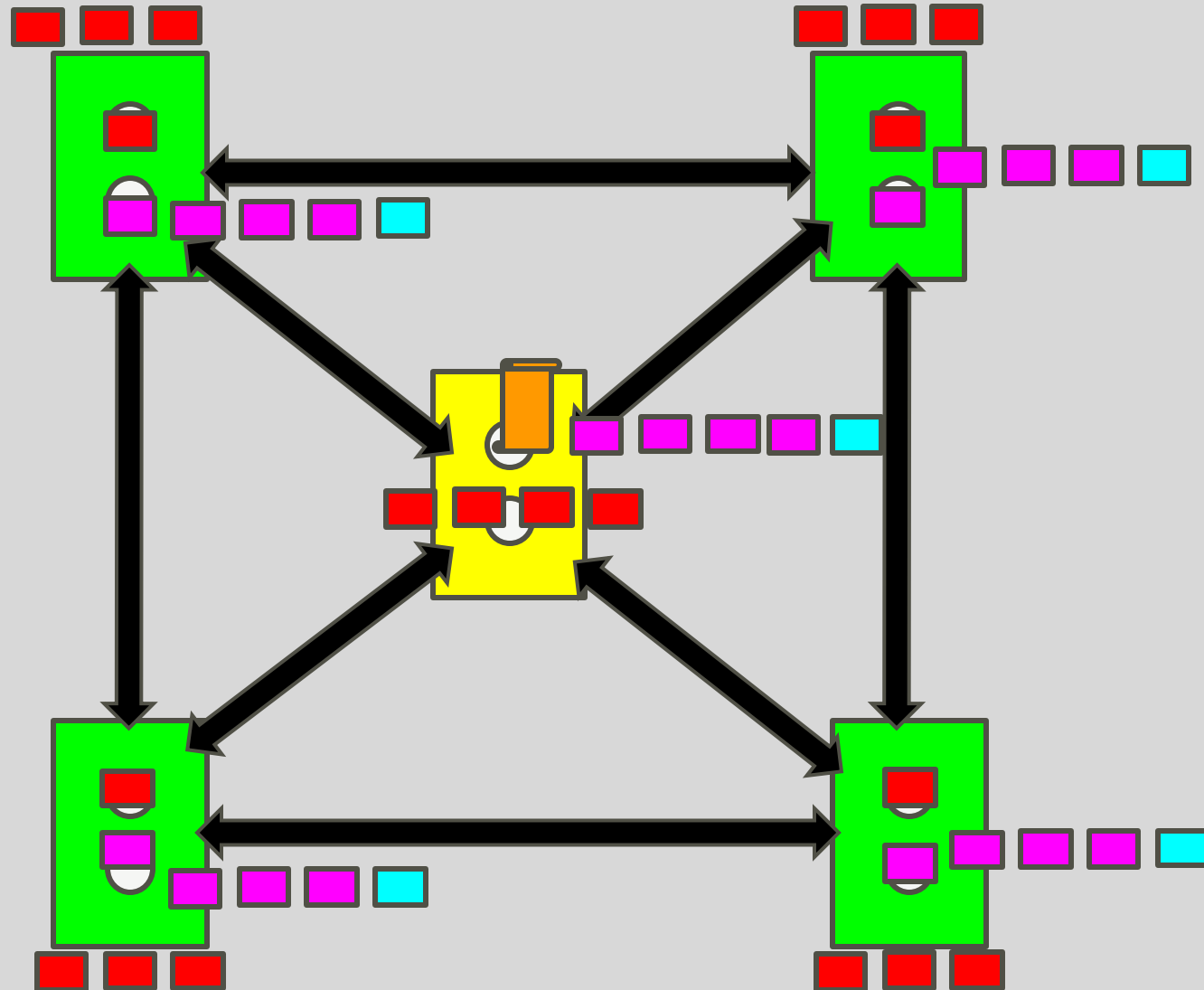


Working of PBFT

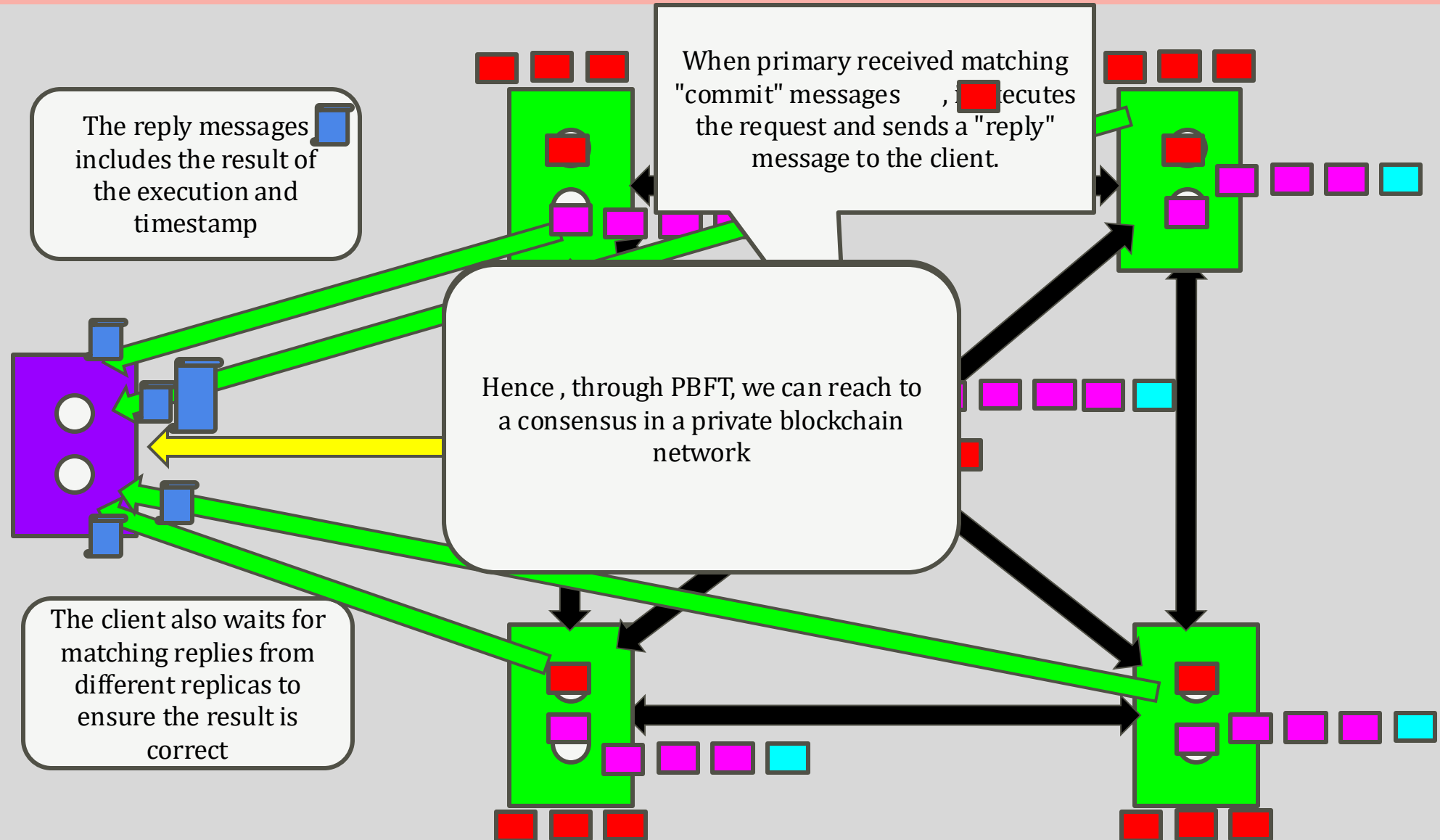
Once a replica has received "prepare messages" (including its own), it broadcasts a "commit message"  to all other replicas including primary

The "commit message" contains the view number 'v' and the sequence number 'n'

Each replica waits to receive matching "commit messages"  for the same sequence number and view



Working of PBFT





Thank You